Proyecto de OpenCV

Rodrigo Rina Mendo

Índice

1. Módulos de la implementación 3
1.1 Captura de la partitura y cambio de colores (intercambiarColores) 3
1.2. Aislar las 5 líneas del pentagrama (aislarLineas) 4
1.3. Extracción de alturas y métricas de escala (obtenerAlturasLineas) 5
1.4. Sustracción del pentagrama y cicatrización (sustraerPentagramaYCicatrizar) 6
1.5. Detección y relleno de notas blancas (rellenarBlancasYRegistrarHuecos) 7
1.6. Aislamiento de cabezas de notas (aislarCabezasNotas) 8
1.7. Validación morfológica y espacia (esNotaValida) 9
1.8. Análisis y clasificación rítmica (clasificarRitmoNota) 10
1.9. Detección orquestada de notas (detectarNotas) 11
1.10. Ubicación espacial precisa de las notas (ubicarNotas) 12
1.11. Extracción del pentagrama local (obtenerLineasCercanas) 13
1.12. Identificación tonal geométrica (identificarNota) 14
1.13. Coloreado y renderizado con transparencia (colorearNotas) 15
1.14. Secuenciación melódica y síntesis sonora (reproducirMelodia) 16
2. Problemas ocurridos y soluciones técnicas 17
2.1. El sesgo por “números mágicos” 17
2.2. Adherencia morfológica de notas al pentagrama 17
2.3. Degradación de notas blancas frente a roturas 17
2.4. Detección de falsos positivos: claves, armaduras y silencios 18
2.5. Inestabilidad milimétrica en la frontera tonal (SOL vs. LA) 18
2.6. Inferencia rítmica errónea en vigas de corcheas y líneas adicionales 18
3. Mejoras futuras y líneas de trabajo no implementadas 19
3.1. Animación sincronizada en tiempo real (seguimiento nota a nota) 19
3.2. Detección de alteraciones tonales (sostenidos, bemoles y becuadros) 19
3.3. Inferencia automática de métrica de compás y tempo adaptativo (BPM) 20

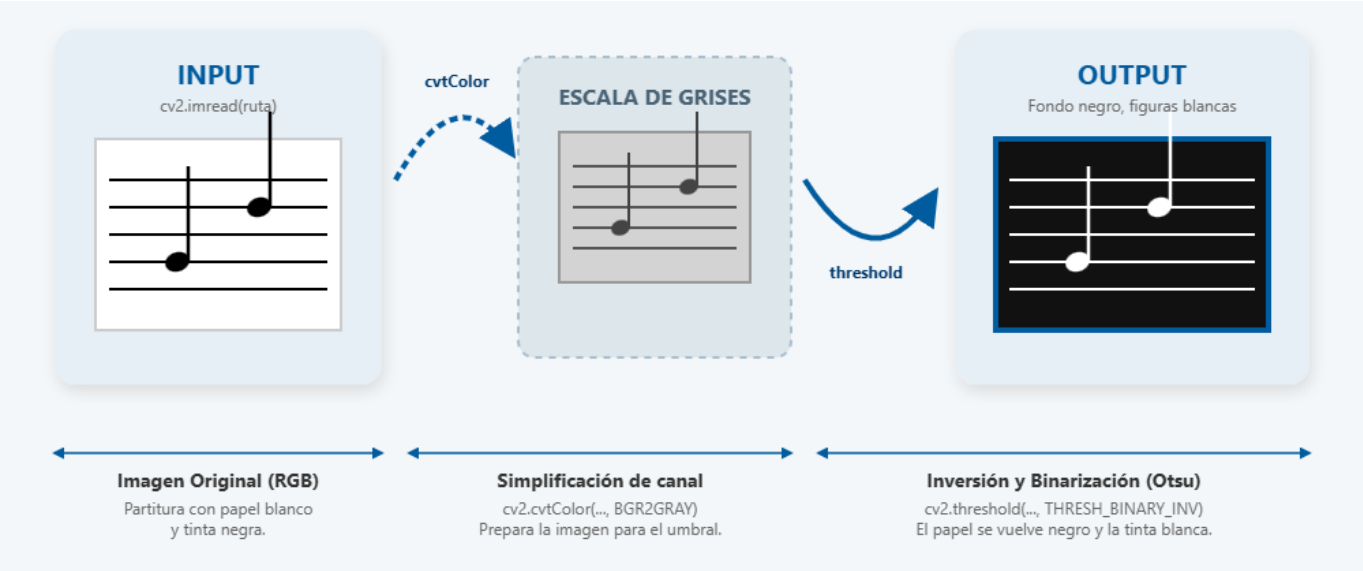
1. Módulos de la implementación

1.1 Captura de la partitura y cambio de colores (intercambiarColores)

Descripción: carga la partitura en formato digital, la convierte a escala de grises y genera una versión binaria invertida donde el papel pasa a ser fondo negro y los elementos musicales (líneas y notas) se transforman en blanco puro para poder analizarlos.

Funciones de OpenCV utilizadas y justificación:

- **cv2.imread(ruta):** carga la imagen de la partitura desde el almacenamiento local a una matriz NumPy en espacio BGR.
- **cv2.cvtColor(img, cv2.COLOR_BGR2GRAY):** reduce la imagen de 3 canales de color a un único canal de luminancia, requisito técnico indispensable para segmentar por luminosidad.
- **cv2.threshold(img_byn, 180, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU):**
    - Se usa la bandera THRESH_BINARY_INV porque las funciones de análisis morfológico y contornos de OpenCV buscan objetos en color blanco (valor 255) sobre fondo negro (valor 0).
    - Se combina con THRESH_OTSU para que el algoritmo calcule automáticamente el umbral óptimo de separación entre tinta y papel, adaptándose a variaciones de iluminación o tono sin fijar valores manuales rígidos.

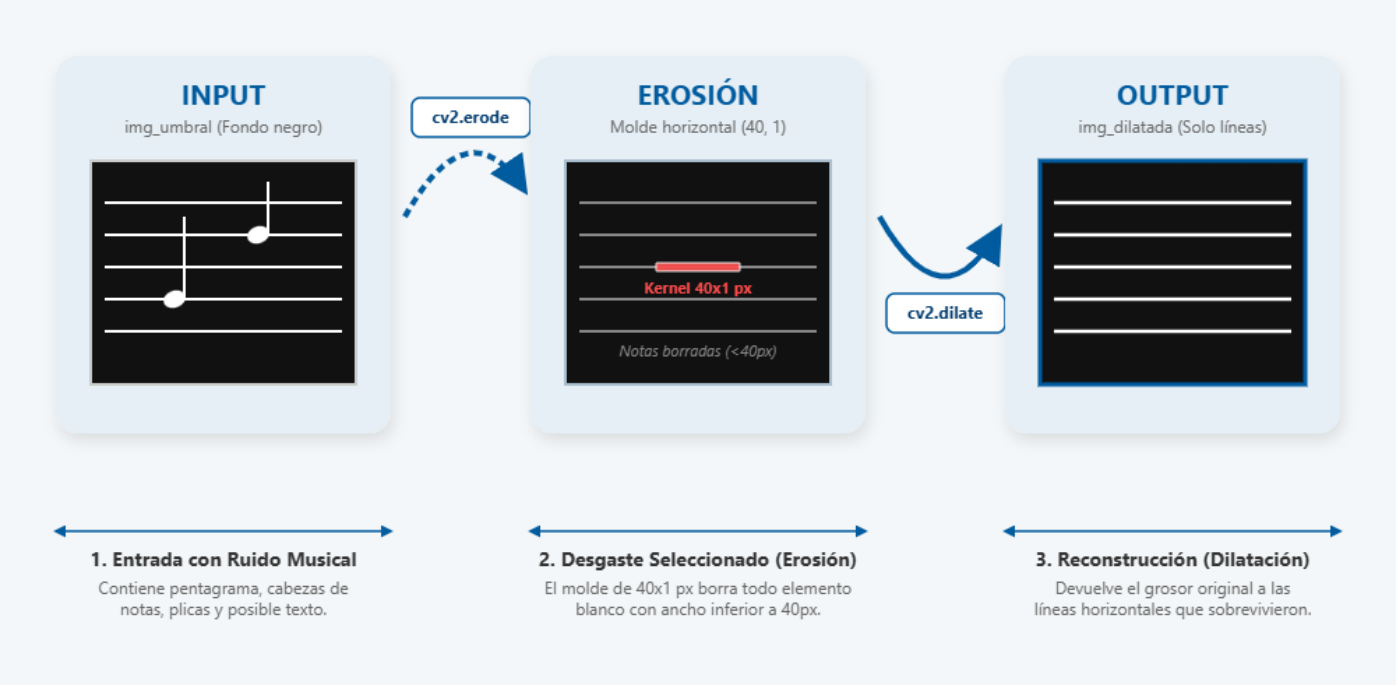


1.2. Aislar las 5 líneas del pentagrama (aislarLineas)

Descripción: filtra la imagen binaria mediante morfología matemática horizontal para eliminar todos los elementos no continuos (cabezas de notas, plicas, texto y figuras musicales), conservando exclusivamente las líneas del pentagrama que servirán de referencia espacial.

Funciones de OpenCV utilizadas y justificación:

- **cv2.getStructuringElement(cv2.MORPH_RECT, (40, 1)):** genera un elemento estructurante (*kernel*) puramente horizontal de 40*1 píxeles, diseñado geométricamente para encajar solo en trazos horizontales largos y descartar cualquier forma vertical o curva.
- **cv2.erode(img_umbral, kernel_horizontal, iterations=1):** desgasta los elementos blancos de la imagen; al aplicar el kernel horizontal largo, cualquier componente cuyo ancho sea inferior a 40 px (como plicas o cabezas de notas) se suprime por completo.
- **cv2.dilate(img_erosionada, kernel_horizontal, iterations=1):** Reconstruye el grosor original de las líneas continuas supervivientes a la erosión previa (operación equivalente a una apertura morfológica, MORPH_OPEN).

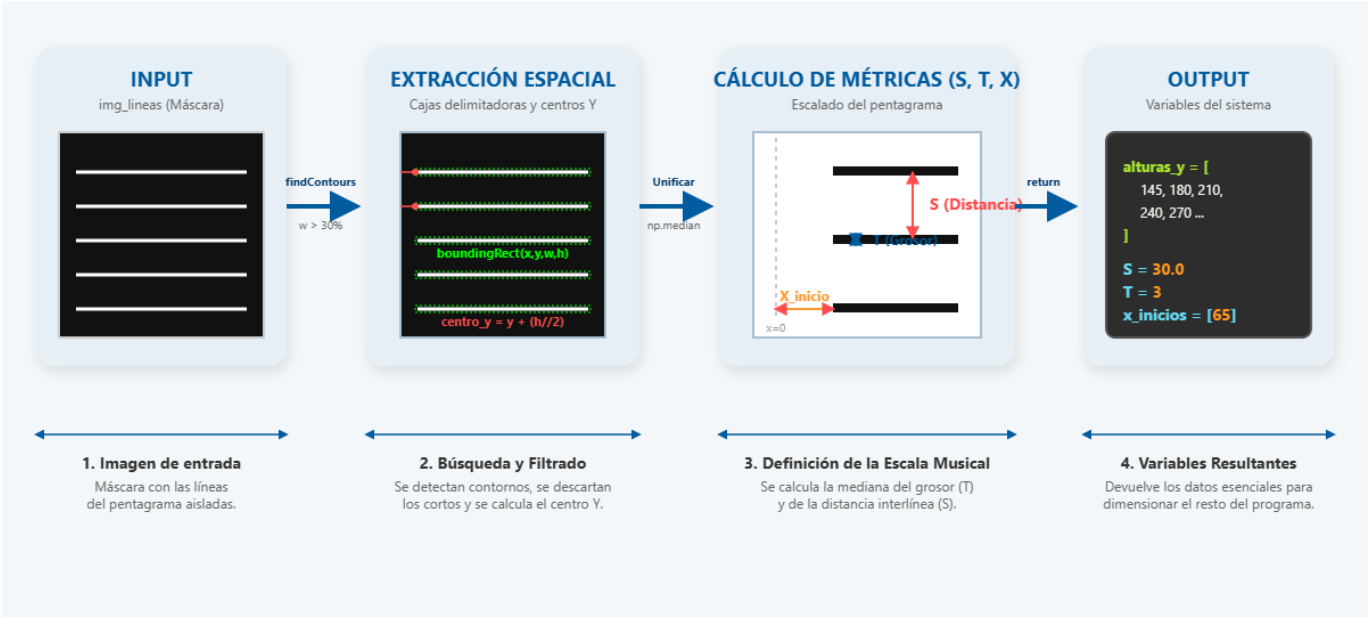


1.3. Extracción de alturas y métricas de escala (obtenerAlturasLineas)

Descripción: localiza verticalmente las líneas del pentagrama para calcular las dos medidas invariantes de la partitura: la distancia interlínea (*S*) y el grosor de trazo (*T*), permitiendo adaptar de forma automática los tamaños de filtros, umbrales y áreas en cualquier imagen sin importar su resolución.

Funciones de OpenCV utilizadas y justificación:

- **cv2.findContours(img_lineas, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE):**
    - Se usa RETR_EXTERNAL para detectar únicamente el perímetro exterior de cada línea continua, ignorando estructuras anidadas.
    - Se aplica CHAIN_APPROX_SIMPLE para comprimir segmentos horizontales y rectos en solo sus puntos extremos, ahorrando memoria y acelerando el cálculo.
- **cv2.boundingRect(contorno):** extrae la caja envolvente (x, y, w, h) de cada línea candidata; permite descartar trazos cortos mediante un ancho mínimo (w > 30% del ancho total), calcular su centroide vertical exacto ($y + h/2$) y registrar el margen izquierdo (x_inicios_pentagramas) donde arranca la partitura.

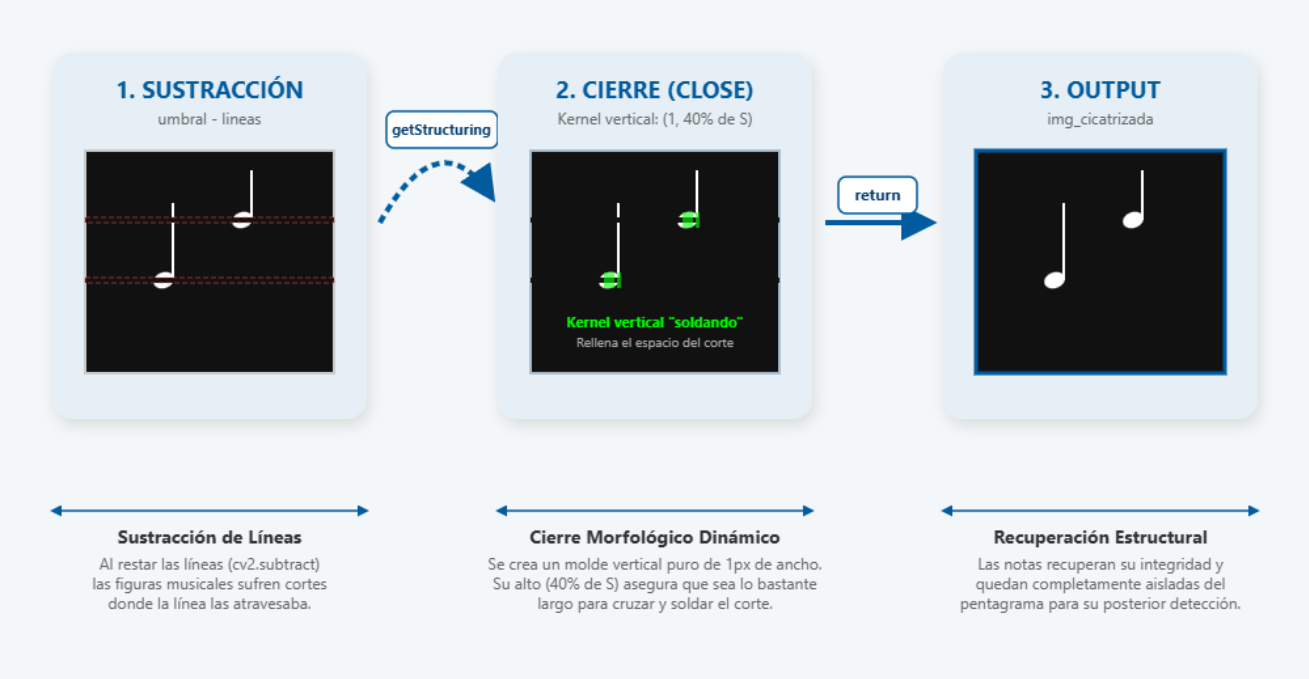


1.4. Sustracción del pentagrama y cicatrización (sustraerPentagramaYCicatrizar)

Descripción: elimina las líneas del pentagrama restándolas de la imagen binaria original y repara («cicatriza») de forma adaptativa las brechas horizontales que dicha sustracción deja dentro de las cabezas y plicas de las notas.

Funciones de OpenCV utilizadas y justificación:

- **cv2.subtract(img_umbral, img_lineas):** realiza una resta píxel a píxel con saturación; elimina el 100% de la tinta de las 5 líneas sin dejar artefactos negativos ni modificar el resto de las figuras musicales que no coincidían con la máscara.
- **cv2.getStructuringElement(cv2.MORPH_RECT, (1, alto_kernel)):** genera un elemento estructurante estrictamente vertical dimensionado dinámicamente según la escala de la partitura (40% de *S*), garantizando altura suficiente para puentear el corte que dejó la línea sin ensanchar los laterales.
- **cv2.morphologyEx(img_sin_lineas, cv2.MORPH_CLOSE, kernel_vertical):** aplica un cierre morfológico (dilatación seguida de erosión) para soldar las fisuras horizontales de las notas y devolverles su continuidad física, conservando su silueta original.

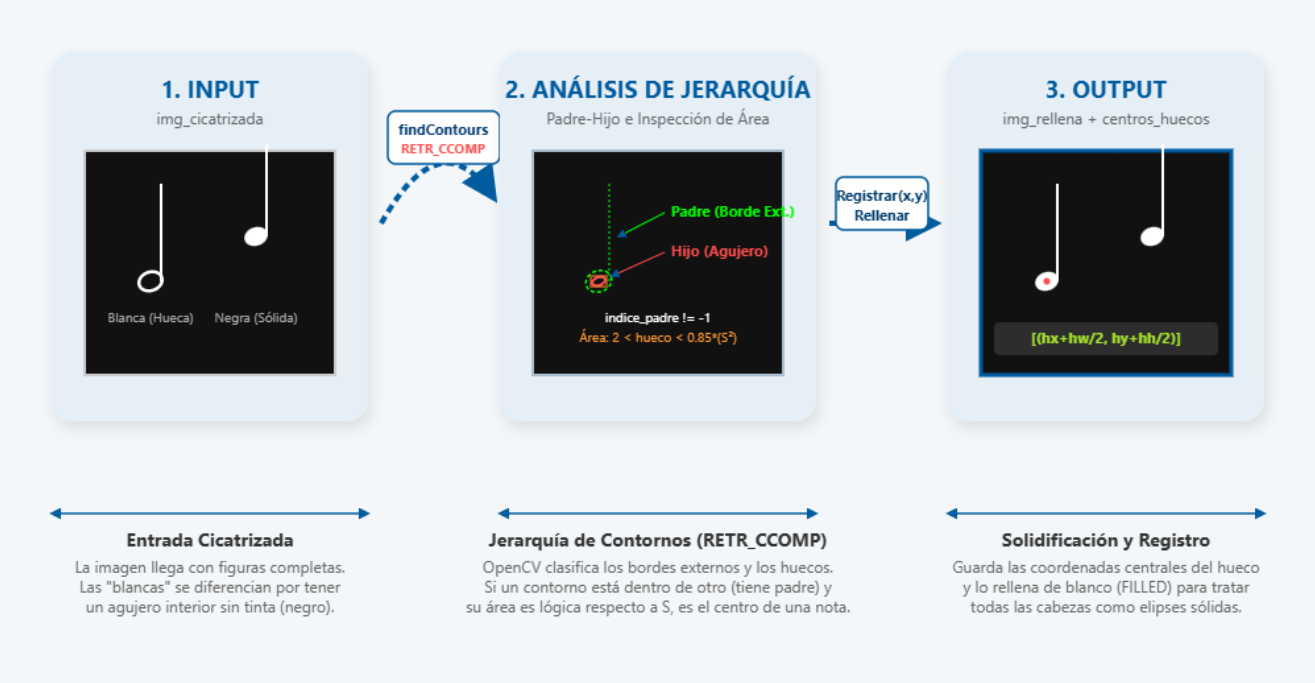


1.5. Detección y relleno de notas blancas (rellenarBlancasYRegistrarHuecos)

Descripción: localiza los huecos interiores característicos de las cabezas de notas blancas (anillos huecos), registra las coordenadas de sus centros para la posterior clasificación rítmica y los rellena para transformarlas en manchas sólidas equivalentes a las negras.

Funciones de OpenCV utilizadas y justificación:

- **cv2.findContours(img_rellena, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE):** organiza los contornos en una jerarquía de dos niveles (componentes externos y huecos interiores); permite aislar los agujeros que tienen un contorno "padre" (indice_padre != -1), descartando el fondo general.
- **cv2.contourArea(contornos[i]):** mide el área en píxeles del hueco interior para descartar ruido diminuto o huecos gigantes no musicales mediante un umbral adaptado a la escala ($0.85 \cdot S^2$).
- **cv2.boundingRect(contornos[i]):** obtiene la posición y tamaño del agujero para calcular su centroide exacto y almacenarlo en la lista de posiciones candidatas a notas blancas.
- **cv2.drawContours(img_rellena, contornos, i, 255, thickness=cv2.FILLED):** rellena de color blanco sólido (255) el interior del contorno del hueco detectado, permitiendo que la blanca resista los filtros morfológicos posteriores sin desintegrarse.

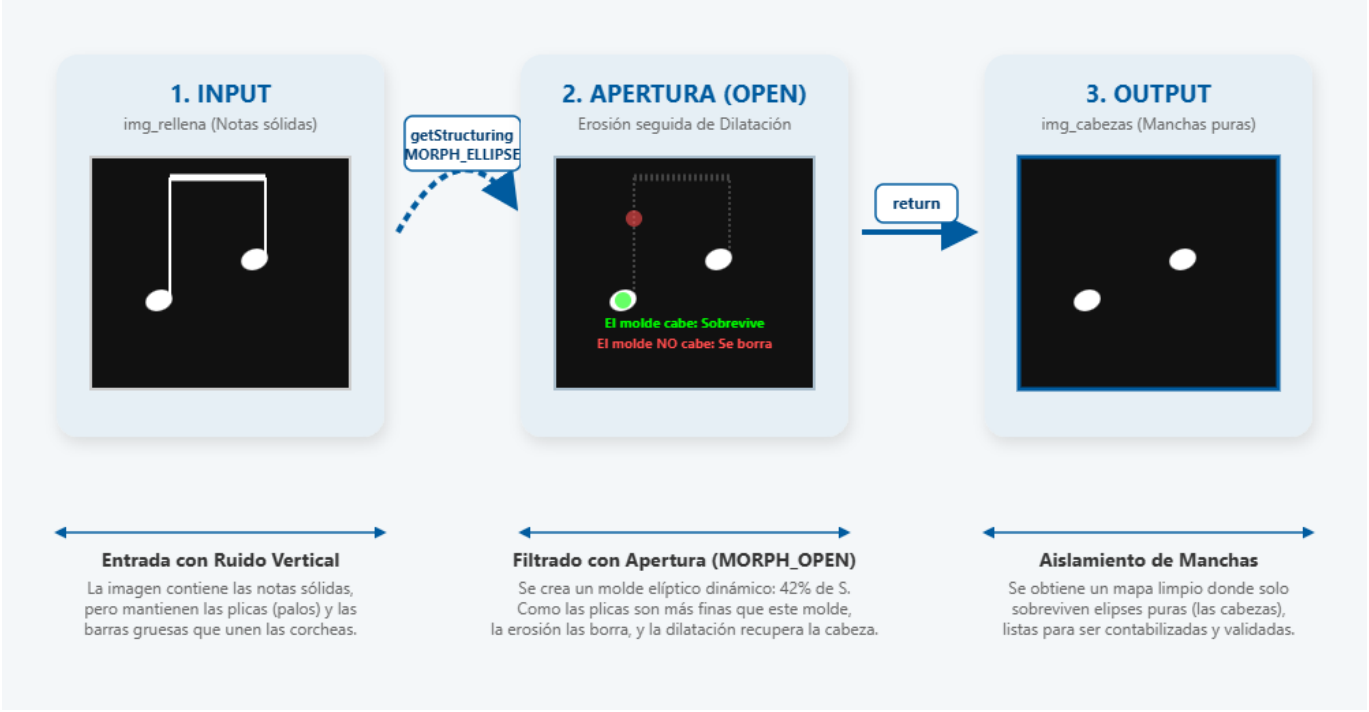


1.6. Aislamiento de cabezas de notas (aislarCabezasNotas)

Descripción: suprime de forma adaptativa las plicas (palos verticales) y las barras de unión de corcheas sobre la imagen ya rellenada, preservando únicamente las manchas elípticas u ovaladas que forman las cabezas de las notas.

Funciones de OpenCV utilizadas y justificación:

- **cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (tamano_molde, tamano_molde)):** Genera un kernel de forma elíptica/circular dimensionado a escala de la partitura (42% de *S*), cuya geometría coincide con la curvatura de la cabeza de una nota, pero es demasiado ancha para caber dentro del trazo fino de una plica vertical.
- **cv2.morphologyEx(img_rellena, cv2.MORPH_OPEN, molde):** ejecuta una apertura morfológica (erosión seguida de dilatación) que disuelve y elimina cualquier trazo blanco más estrecho que el molde (palos, barras y líneas auxiliares finas), restaurando intacta la masa central de las cabezas.



1.7. Validación morfológica y espacia (esNotaValida)

Descripción: discrimina mediante restricciones geométricas y espaciales si un contorno candidato corresponde realmente a una cabeza de nota o si debe descartarse por tratarse de ruido, texto, silencios o la clave de Sol.

Funciones de OpenCV utilizadas y justificación:

- **cv2.contourArea(contorno):** calcula el área en píxeles del candidato para validar que su tamaño esté dentro de los límites anatómicos de una cabeza de nota ($0.34 \cdot S^2 < \acute{a}rea < 2.20 \cdot S^2$), eliminando motas de polvo o borrones grandes.
- **cv2.boundingRect(contorno):**
  - Proporciona ancho (w), alto (h) y posición (x, y)para calcular tanto la relación de aspecto (w/h) como la solidez ($\acute{a}rea/(w \cdot h)$), descartando figuras irregulares o no ovaladas.
  - Permite validar que el centro vertical de la figura esté contenido dentro del margen vertical del pentagrama ($\pm 1.5 \cdot S$) y que su posición horizontal supere el margen inicial de la clave de Sol (x > margen_x).

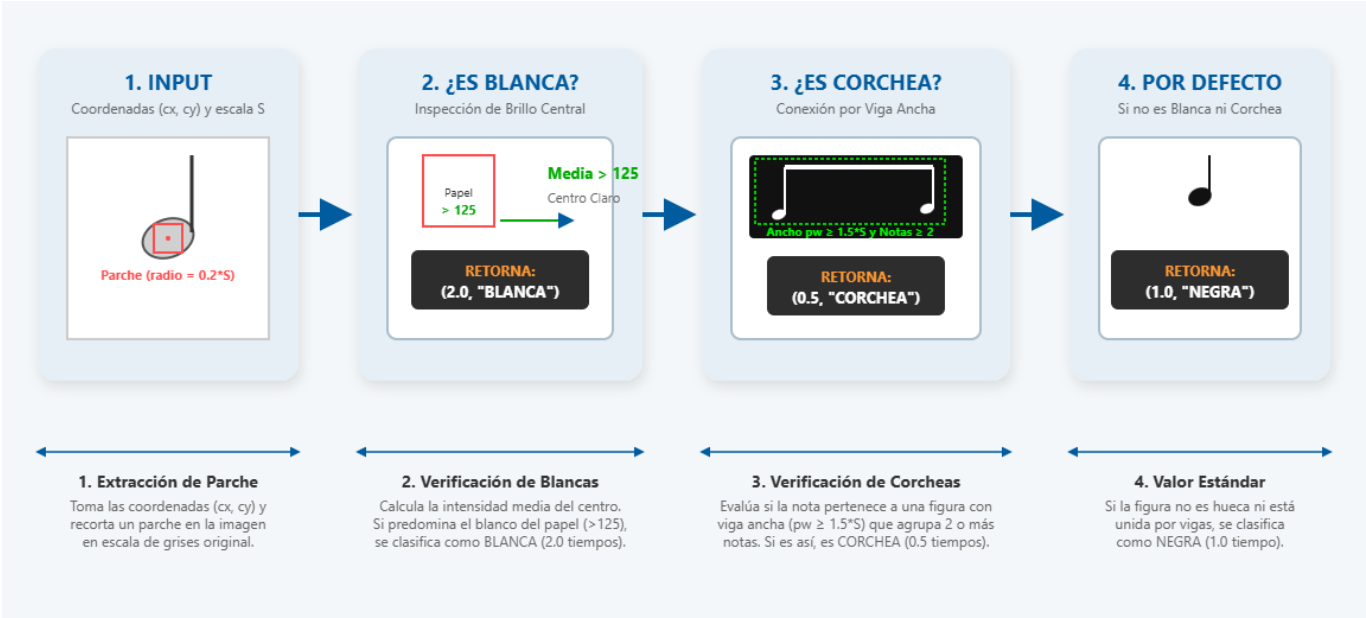


1.8. Análisis y clasificación rítmica (clasificarRitmoNota)

Descripción: determina la figura musical (blanca, negra o corchea) y asigna su duración rítmica en tiempos comprobando el nivel de tinta en el núcleo de la nota o su conexión estructural con otras figuras mediante barras de corchea.

Funciones de OpenCV utilizadas y justificación:

- *Nota de indexación directa (NumPy):* Extrae una región de interés (ROI) circular de radio $0.20 \cdot S$sobre la imagen en escala de grises centrada en (c, xcy); si el promedio de brillo supera 125 denota presencia de papel blanco en su interior (nota blanca = 2 tiempos), si no, es tinta maciza (negra = 1 tiempo).
- **cv2.boundingRect(c_palo):** Mide las cajas delimitadoras de los trazos complejos de la imagen cicatrizada; si el ancho del bloque envolvente supera $1.5 \cdot S$y engloba dos o más centros de notas en su interior, identifica una viga horizontal de unión catalogando las figuras como corcheas (0.5 tiempos).

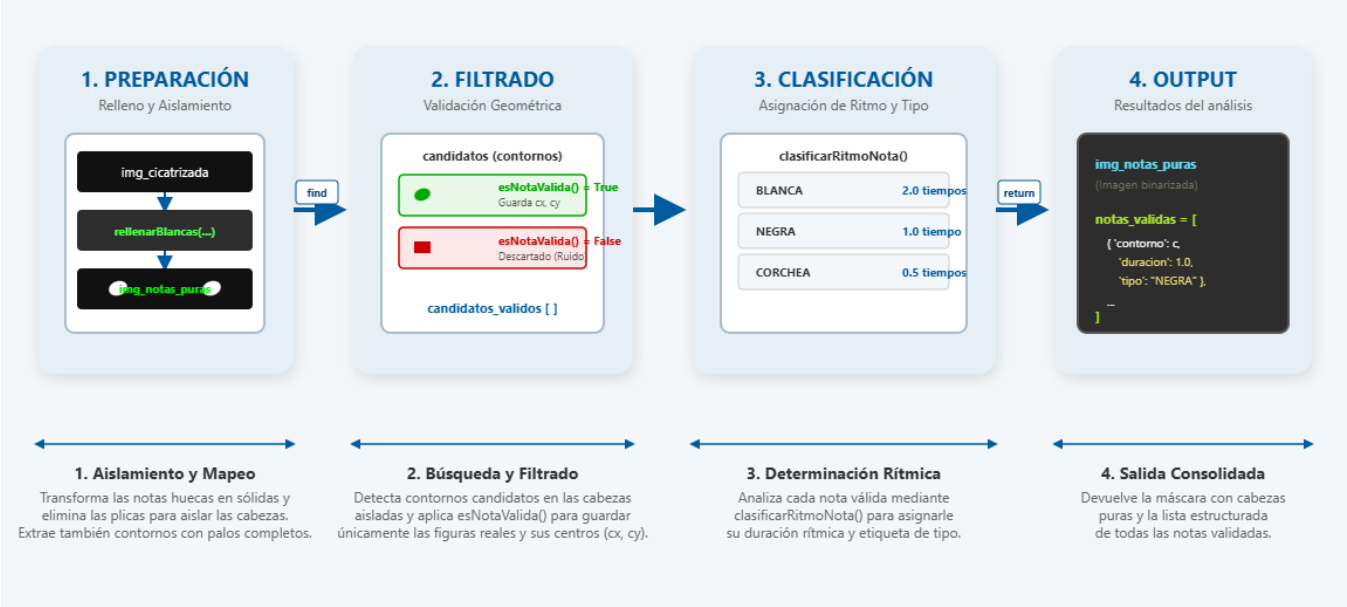


1.9. Detección orquestada de notas (detectarNotas)

Descripción: actúa como pipeline coordinador: ejecuta el rellenado de huecos, aísla las cabezas, detecta los contornos candidatos sobre la imagen procesada y sobre la imagen con plicas, filtrando y asignando el ritmo definitivo a cada nota válida.

Funciones de OpenCV utilizadas y justificación:

- **cv2.findContours(img_notas_puras, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)**: escanea la imagen donde solo sobrevivieron las manchas elípticas para extraer la lista de contornos candidatos que pasarán por el filtro de validación.
- **cv2.findContours(img_cicatrizada, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE):** escanea de forma paralela la imagen con plicas y barras para obtener la geometría estructural completa necesaria en la detección de agrupaciones de corcheas.



1.10. Ubicación espacial precisa de las notas (ubicarNotas)

Descripción: calcula el centro geométrico de cada cabeza de nota detectada con precisión subpíxel, dibuja los marcadores de depuración visual sobre la partitura y ordena cronológicamente las figuras musicales de izquierda a derecha para su lectura secuencial.

Funciones de OpenCV utilizadas y justificación:

- **cv2.boundingRect(contorno):** obtiene la caja delimitadora (x, y, w, h) para enmarcar la nota y servir como cálculo de centroide alternativo en caso de contornos con menos de 5 puntos.
- **cv2.fitEllipse(contorno):** ajusta una elipse matemática a los puntos del perímetro de la cabeza de la nota; proporciona el centroide exacto (cx, cy) neutralizando posibles tocones residuales del palo o irregularidades en el borde que desviarían el cálculo de la nota hacia una línea o espacio adyacente.
- **cv2.rectangle(img_resultado, (x, y), (x + w, y + h), (0, 255, 0), 2):** dibuja el marco verde de diagnóstico alrededor de cada nota validada para verificar visualmente los límites de la segmentación.
- **cv2.circle(img_resultado, (cx, cy), 3, (0, 0, 255), -1):** pinta un punto rojo sólido en el centroide calculado de la nota para certificar la posición exacta que se comparará contra las líneas del pentagrama.



1.11. Extracción del pentagrama local (obtenerLineasCercanas)

Descripción: agrupa las líneas detectadas en bloques de 5 (un pentagrama por bloque) y asigna de forma dinámica a cada nota su pentagrama de referencia específico según su coordenada vertical cy.

Funciones de OpenCV utilizadas y justificación:

- ***Nota de diseño e indexación algorítmica:*** no requiere llamadas directas a la API de OpenCV. Opera sobre la lista de alturas *Y* previamente extraídas con cv2.findContours:
    - Segmenta la lista total de alturas en sublistas ordenadas de tamaño 5 (alturas_pentagrama[i:i+5]).
    - Evalúa si el centroide de la nota cae dentro de la franja vertical comprendida entre la línea superior (pentagrama[0] - 50 px) y la línea inferior (pentagrama[4] + 50 px) del bloque evaluado, garantizando que una partitura con múltiples sistemas no confunda las líneas de un pentagrama con las del siguiente.



1.12. Identificación tonal geométrica (identificarNota)

Descripción: calcula la distancia vertical entre el centro de la nota y las referencias del pentagrama asignado, determinando primero si descansa sobre una línea física o asociándola al escalón o espacio diatónico más cercano para nombrar la nota (DO a SOL5).

Funciones de OpenCV utilizadas y justificación:

- ***Nota de diseño e inferencia matemática:*** prescinde de procesado de imagen en OpenCV para evitar errores de discretización por píxeles y opera mediante cálculo de distancias euclídeas directas:
    - Aplica un radio de tolerancia de captura prioritario ($0.20 \cdot$ distancia_media) sobre las 5 líneas reales (L_0 a L_4) para asegurar la detección de notas que pisan línea (Mi, Sol, Si, Re5, Fa5).
    - Si la nota no cae en el margen de atracción de una línea, genera el mapa de alturas teóricas de los espacios y líneas adicionales (-2 a 9) y selecciona el escalón óptimo evaluando la distancia absoluta mínima ($\min |\, y_{teórico} - cy \,|$).

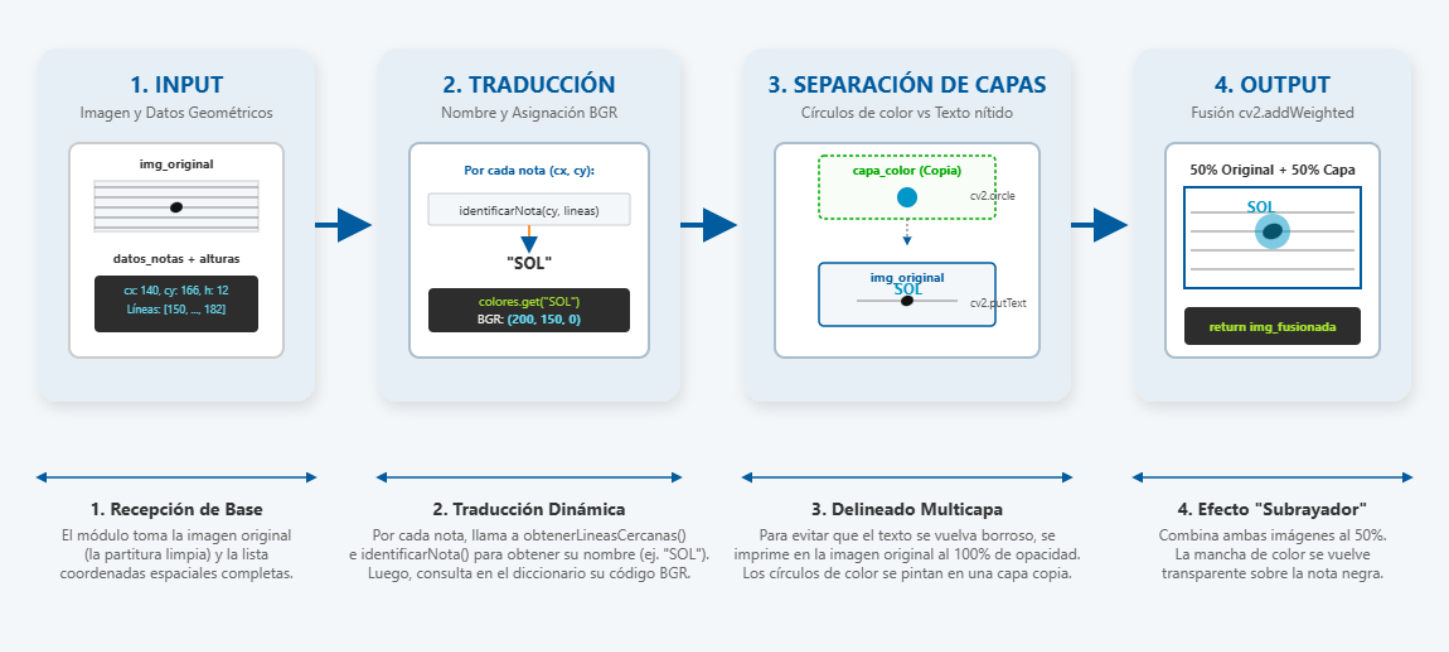


1.13. Coloreado y renderizado con transparencia (colorearNotas)

Descripción: superpone una máscara de color translúcida sobre cada nota según su tono musical e imprime su nombre en texto claro, logrando un efecto visual de "resaltador fluorescente" que no tapa la tinta original de la partitura.

Funciones de OpenCV utilizadas y justificación:

- **cv2.circle(capa_color, (int(cx), int(cy)), radio, color_nota, -1):** dibuja sobre una capa auxiliar un círculo sólido relleno (*thickness* = -1) con el color asignado a la nota musical en formato BGR (ej. rojo para DO, azul para SOL).
- **cv2.putText(img_original, nombre_nota, (int(cx) - 15, int(cy) - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.4, color_nota, 1):** escribe la etiqueta textual de la nota ("DO", "FA", "LA") directamente sobre la partitura para garantizar su legibilidad nítida sin verse afectada por el difuminado de transparencias.
- **cv2.addWeighted(capa_color, 0.5, img_original, 0.5, 0):** realiza una suma lineal ponderada entre la imagen original y la capa de círculos ($Salida = 0.5 \cdot Capa + 0.5 \cdot Original + 0$), integrando los colores con un 50% de transparencia para conservar visibles las notas y el pentagrama de fondo.

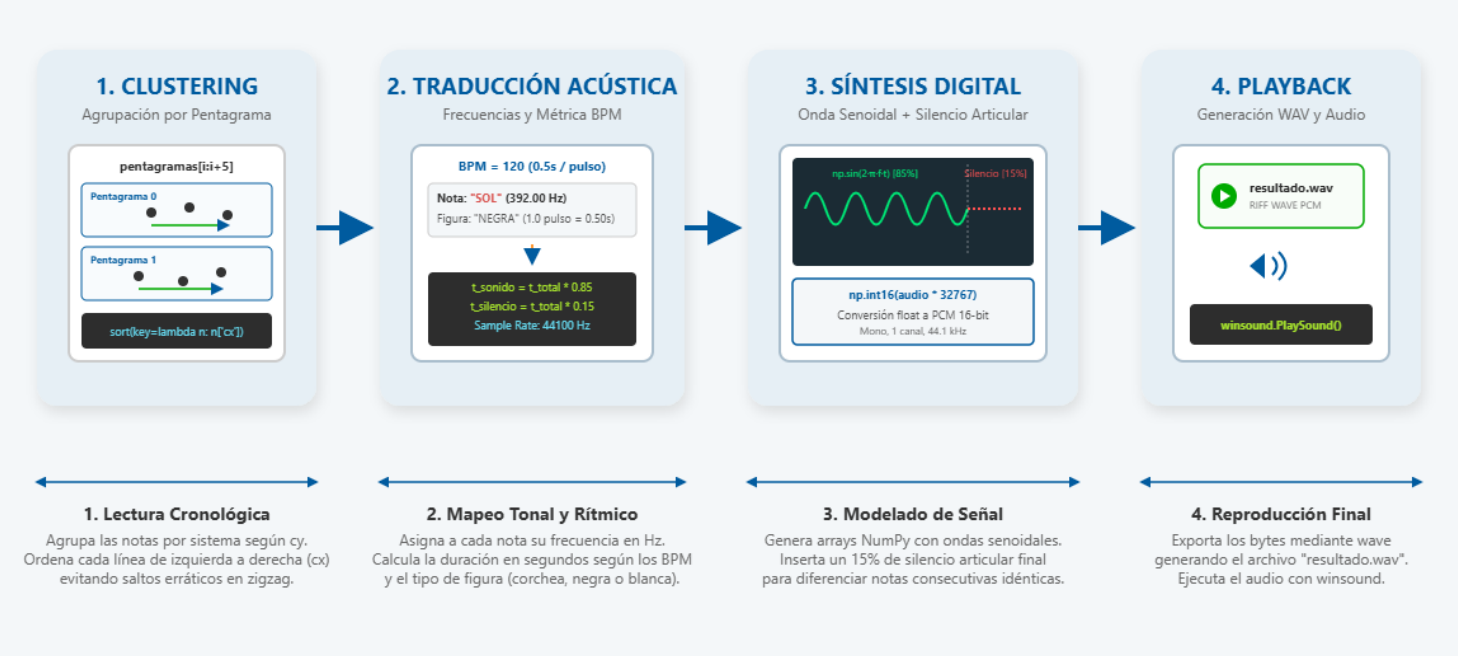


1.14. Secuenciación melódica y síntesis sonora (reproducirMelodia)

Descripción: agrupa las notas por pentagrama, las ordena cronológicamente de izquierda a derecha y genera un archivo de audio WAV sintetizado donde cada nota suena con su frecuencia exacta en hercios y la duración temporal correspondiente a su figura rítmica (a tempo configurable en BPM).

Funciones de OpenCV utilizadas y justificación:

- ***Nota de diseño e integración acústica:*** este módulo culmina el pipeline de visión por computador y traslada los datos espaciales al dominio acústico. No requiere llamadas a la API visual de OpenCV, sino el uso de librerías numéricas y de audio de Python:
    - **numpy.linspace y numpy.sin:** sintetizan la onda senoidal pura para cada frecuencia (Hz) a una tasa estándar de 44.1 kHz, aplicando una proporción de 85% sonido y 15% silencio para separar con claridad las notas consecutivas.
    - **wave y winsound.PlaySound:** serializan el flujo de datos a PCM de 16 bits, lo empaquetan en un archivo *.wav* físico y disparan la reproducción sonora a través del hardware del sistema.



2. Problemas ocurridos y soluciones técnicas

2.1. El sesgo por "números mágicos"

Explicación del problema: el sistema utilizaba inicialmente umbrales rígidos en píxeles (áreas entre 120 y 400 px^2, márgenes fijos $x > 80$y filtros de 5×5). Aunque funcionaba en una partitura de prueba, fallaba por completo al procesar otras con diferente densidad de píxeles (DPI), ya que una distancia interlínea pequeña ($S = 9$ px) producía notas diminutas de 60px^{2}que el filtro descartaba como ruido, mientras que en partituras grandes ($S = 20$ px) dejaba pasar artefactos gigantes.

Solución implementada: se adoptó la distancia interlínea (*S*) mediana de la separación entre líneas consecutivas) como la unidad de medida canónica e invariante de escala del proyecto. Todos los parámetros morfológicos, áreas válidas ($0.34 \cdot S^2 <$ área $< 2.20 \cdot S^2$), dimensiones de kernels y radios de tolerancia se redefinieron como múltiplos o fracciones directas de *S*, logrando que el pipeline se autoajuste dinámicamente a cualquier resolución de imagen.

2.2. Adherencia morfológica de notas al pentagrama

Explicación del problema: al intentar segmentar las notas directamente sobre la partitura intacta, las líneas horizontales permanecían fusionadas a las cabezas elípticas formando rebabas laterales. Esta distorsión ensanchaba anormalmente la caja delimitadora (*bounding box*), hundiendo la métrica de solidez geométrica (área$/(w \cdot h)$) e impidiendo diferenciar matemáticamente una nota real sobre una línea de un silencio o una barra de compás.

Solución implementada: se diseñó un procedimiento de sustracción y cicatrización morfológica: primero se sustrajo algebraicamente la máscara pura de líneas mediante cv2.subtract, aislando las notas pero dejando un corte horizontal en su cuerpo. A continuación, se aplicó un cierre morfológico (MORPH_CLOSE) con un elemento estructurante estrictamente vertical de 1pxde ancho, incapaz de reconstruir las líneas horizontales pero idóneo para soldar la brecha vertical de las cabezas en el vacío.

2.3. Degradación de notas blancas frente a roturas

Explicación del problema: las notas blancas (anillos huecos) colapsaban ante la morfología binaria: si se usaba un *kernel* vertical grande para soldar cortes en partituras de alta resolución, este dilataba hacia el interior y cegaba el agujero de la blanca convirtiéndola en negra en partituras pequeñas ($S = 9$px). Si por el contrario se reducía el *kernel* a 3pxpara preservar el hueco, las notas atravesadas por líneas no se cerraban, partiéndose en dos mitades que la posterior apertura morfológica borraba por completo.

Solución implementada: se desacopló la reconstrucción geométrica de la clasificación rítmica: se aplicó un *kernel* de cicatrización fuerte adaptado a la escala ($0.40 \cdot S$) para garantizar la supervivencia de todas las notas sin temor a cegar el hueco binario. Para catalogar la figura, una vez localizado el centroide (c, xcy), se inspeccionó directamente

la reflectancia en la imagen original en escala de grises: un promedio de brillo en el núcleo superior a 125 identifica inequívocamente el papel blanco interior de una blanca, mientras que la tinta sólida de una negra arroja promedios oscuros.

2.4. Detección de falsos positivos: claves, armaduras y silencios

Explicación del problema: elementos estructurales ajenos a las notas activaban detecciones erróneas: el gancho inferior de la clave de Sol coincidía en morfología y altura con un DO central, las roturas morfológicas de números de compás (como el '3' en 3/4) simulaban notas DO5, y el remate curvo de los silencios de negra quedaba aislado tras la erosión generando notas fantasma.

Solución implementada: se estableció un sistema de filtrado multicapa: se calculó el inicio horizontal real de cada pentagrama (x_inicios_pentagramas) aplicando una exclusión espacial adaptada ($5.2 \cdot S$en el primer sistema para saltar clave y compás, y $3.2 \cdot S$en los siguientes). Asimismo, los residuos de silencios y números rotos se erradicaron imponiendo restricciones estrictas de relación de aspecto, ancho mínimo ($w \geq 0.55 \cdot S$) y un umbral de solidez superior a 0.52.

2.5. Inestabilidad milimétrica en la frontera tonal (SOL vs. LA)

Explicación del problema: en figuras de baja resolución, la distancia entre una línea y su espacio adyacente es de apenas $S/2$(4 a 5 píxeles). Pequeñas asimetrías de un solo píxel en el contorno, provocadas por tocones residuales de la plica tras la erosión, desplazaban el centroide calculado mediante momentos clásicos (cv2.moments), provocando lecturas caóticas alternadas entre línea y espacio (ej. SOL ->LA -> SOL) en notas idénticas.

Solución implementada: se sustituyó el cálculo de masa por el ajuste algebraico de elipses (cv2.fitEllipse) sobre el perímetro de la cabeza, neutralizando la influencia de los apéndices verticales y fijando el centroide anatómico exacto. Además, se introdujo una franja de atracción gravitatoria en identificarNota ($0.20 \cdot S$) sobre cada una de las 5 líneas físicas), priorizando la asignación a la línea real si el centroide cae dentro de su margen de tolerancia.

2.6. Inferencia rítmica errónea en vigas de corcheas y líneas adicionales

Explicación del problema: la identificación de corcheas basada únicamente en el ancho horizontal de la caja delimitadora generaba falsos positivos críticos: notas aisladas como el DO central eran clasificadas como corcheas debido a que su línea adicional horizontal atravesada aumentaba artificialmente el ancho del contorno hasta los 20 píxeles.

Solución implementada: se formuló una regla topológica de agrupamiento en clasificarRitmoNota: una estructura con plica se clasifica como corchea (0.5 pulsos) si y solo si su caja delimitadora presenta dimensiones de viga ($pw \geq 1.5 \cdot S$) y contiene simultáneamente dos o más centros de notas en su interior. Toda figura aislada que no comparta viga con otra nota pasa automáticamente a clasificarse como negra (1.0 pulso) o blanca según su análisis de brillo interior.

3. Mejoras futuras y líneas de trabajo no implementadas

3.1. Animación sincronizada en tiempo real (seguimiento nota a nota)

Explicación del reto técnico: el objetivo inicial era iluminar dinámicamente cada nota con su círculo de color en el fotograma en el instante exacto en que comenzaba a sonar, creando una experiencia visual interactiva donde el cursor gráfico saltase al ritmo de la música a lo largo de la partitura.

Complejidad y causa de descarte: lograr este comportamiento exige desacoplar el procesamiento de la imagen y la emisión de audio mediante programación concurrente (hilos de ejecución o *multithreading* asíncrono). Si se ejecuta de forma síncrona en el bucle principal, funciones bloqueantes como time.sleep() o la reproducción de audio congelan la ventana de refresco de OpenCV (cv2.waitKey), deteniendo la interfaz gráfica a 0 FPS durante la duración de la nota; si por el contrario se reduce el tiempo de espera, el audio se desincroniza del refresco gráfico acumulando desfases notables (*jitter* y latencia acumulada). Ante esta complejidad de sincronismo temporal entre hardware de audio y renderizado gráfico, se optó por una arquitectura más robusta y desacoplada: colorear toda la partitura de forma integral con transparencia (cv2.addWeighted) y, a continuación, sintetizar y reproducir el archivo de audio continuo de una sola vez.

3.2. Detección de alteraciones tonales (sostenidos, bemoles y becuadros)

Explicación del reto técnico: se planteó dotar al sistema de reconocimiento tonal avanzado para detectar accidentes musicales (♯, ♭, ♮) tanto en la armadura inicial como a la izquierda inmediata de cada cabeza de nota, alterando matemáticamente la frecuencia generada (sumando o restando un semitono, es decir, multiplicando por $2^{\pm 1/12}$).

Complejidad y causa de descarte: a diferencia de las notas, que poseen una masa ovalada continua y compacta fácil de aislar con morfología matemática, las alteraciones presentan una tipografía extremadamente fina, discontinua y angulada. Al aplicar las etapas de sustracción de líneas del pentagrama (cv2.subtract), las líneas horizontales de

los sostenidos y el cuerpo inferior del bemol quedaban totalmente fragmentados o destruidos. Reconocer estos símbolos exigiría implementar técnicas avanzadas de coincidencia de patrones (*template matching*, cv2.matchTemplate) invariantes a la escala, o entrenar clasificadores de aprendizaje profundo (como redes neuronales convolucionales o Haar-Cascades), lo cual excedía ampliamente el alcance de visión por computador clásica basado en morfología y contornos establecido para el proyecto.

3.3. Inferencia automática de métrica de compás y tempo adaptativo (BPM)

Explicación del reto técnico: la propuesta consistía en leer automáticamente la indicación de compás al inicio de la partitura (por ejemplo, 3/4, 4/4 o 6/8) y marcas de tempo para calibrar los pulsos por minuto (BPM) de forma autónoma, adaptando la velocidad del sintetizador al carácter original de cada obra.

Complejidad y causa de descarte: la segmentación e interpretación de dígitos apilados verticalmente en una partitura presenta una alta ambigüedad visual. En la cabecera, los números de compás se encuentran cruzados por las líneas del pentagrama y muy próximos a la clave de Sol; la rotura provocada por la sustracción de líneas suele partir dígitos como el '3' o el '4' en fragmentos inconexos que el sistema confunde con restos de silencios o cabezas de notas. Además, interpretar fracciones complejas requeriría integrar un motor de reconocimiento óptico de caracteres (OCR) específico para fuentes musicales, además de un análisis de las barras de compás para verificar que la suma rítmica de pulsos por compás sea consistente. Ante esta sobrecarga algorítmica, se prefirió parametrizar el tempo mediante una constante configurable en el código (BPM = 120), garantizando estabilidad sin comprometer el reconocimiento de las notas.